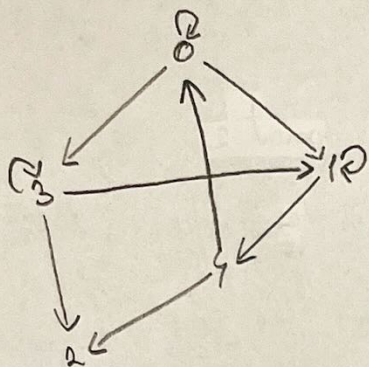


Practical work no 2.

→ Find a lowest length path between two given vertices, by using a backward breadth-first search from the ending vertex.



• the in-edges dictionary

Key	Val
0	- [0, 4]
1	- [1, 0, 3]
2	- [3, 4]
3	- [0, 3]
4	- [1]

I The path exists

We want the shortest path from 1 to 3 $\Rightarrow$ g.shortest-path(^{start}1, ^{end}3)
 In dict tree we store the tree returned by the reverse BFS starting in the ending vertex $\Rightarrow$ tree = bfs(3)

Now we compute bfs(3)

initialization	index	current	queue(q)	parent
	0	-	[end]=[3]	{(end)3: None}
iter 1	0	q[0]=3	[3, 0]	{3: None, 0: 3}
iter 2	1	q[1]=0	[3, 0, 4]	{3: None, 0: 3, 4: 0}
iter 3	2	q[2]=4	[3, 0, 4, 1]	{3: None, 0: 3, 4: 0, 1: 4}
iter 4	3	q[3]=1	all inbounds of 1 are already in the queue	{3: None, 0: 3, 4: 0, 1: 4}

$\Rightarrow$ tree = {3: None, 0: 3, 4: 0, 1: 4}

We check if the keys in tree contain the starting vertex. It is $\Rightarrow$

We construct the path by walking from start to end in the tree dict.

$\Rightarrow$ path [1, 4, 0, 3]

II The path does not exist

We want the shortest path from 2 to 3 $\Rightarrow$ shortest-path(2,3)

We compute bfs(3)

-1- same

$\Rightarrow$ tree = { 3:None, 0:3, 4:0, 1:4 }

We check if the keys in tree contain the starting vertex.

2 is not one of the keys in tree $\Rightarrow$ so a path from 2 to 3 does not exist and the method will return None

Minimum length path from 1 to 100

graph1k.txt: [1, 5, 487, 175, 699, 624, 100] $l=6$

graph10k.txt: [1, 7317, 4118, 2904, 690, 1454, 739, 4722, 100] $l=8$

graph100k.txt: [1, 17024, 27471, 14969, 3075, 4156, 32753, 14973, 100] $l=8$

Minimum length path from 100 to 1

graph1k.txt: [100, 416, 354, 865, 109, 1] $l=5$

graph10k.txt: [100, 5568, 2781, 1451, 4997, 528, 4260, 1] $l=7$

graph100k.txt: [100, 44340, 54527, 6606, 53263, 95930, 98655, 58288, 1] $l=8$